

Digital Design

FSM Encoding Optimization

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

<http://jatinga.iitg.ac.in/~asahu/cs221/>

Outline

- FSM Optimization : State Encoding
 - Binary/Sequential, Gray, One-hot
 - Heuristic Based
- Demerits of FSM Modeling

State Encoding/Assignment

State assignment

- State encoding:
 - Assigning unique binary value to each state
 - So that we can implement FSM
- Since we don't care about the actual flip-flop values for each state
- We can assign each state to any binary number we like **as long as**
 - Each state is assigned a unique binary number

State Encoding Example

- Four states
S0, S1, S2, S3
- 4! options

SL	S0	S1	S2	S3
1	00	01	10	11
2	00	01	11	10
3	00	10	01	11
4	00	10	11	01
5	00	11	01	10
6	00	11	10	01
7	01	00	11	10
8	01	00	10	11
9	01	10	11	00
10	01	10	00	11
11	01	11	10	00
12	01	11	00	11

SL	S0	S1	S2	S3
13	10	01	00	11
14	10	01	11	00
15	10	00	01	11
16	10	00	11	01
17	10	11	01	00
18	10	11	00	01
19	11	00	01	10
20	11	00	10	01
21	11	10	01	00
22	11	10	00	01
23	11	01	10	00
24	11	01	00	11

State assignment

- Since we don't care about the actual flip-flop values for each state we can assign each state to any binary number we like **as long as** each state is assigned a unique binary number
- Suppose a FSM have 6 states: Modulo 6 Counter
- If we use 3 bits to encode the 6 states, we have

$$\binom{8}{6} = \frac{8!}{6!(8-6)!}$$

possible choosing the 6 states from 8 total state

- Than encodings

State Encoding

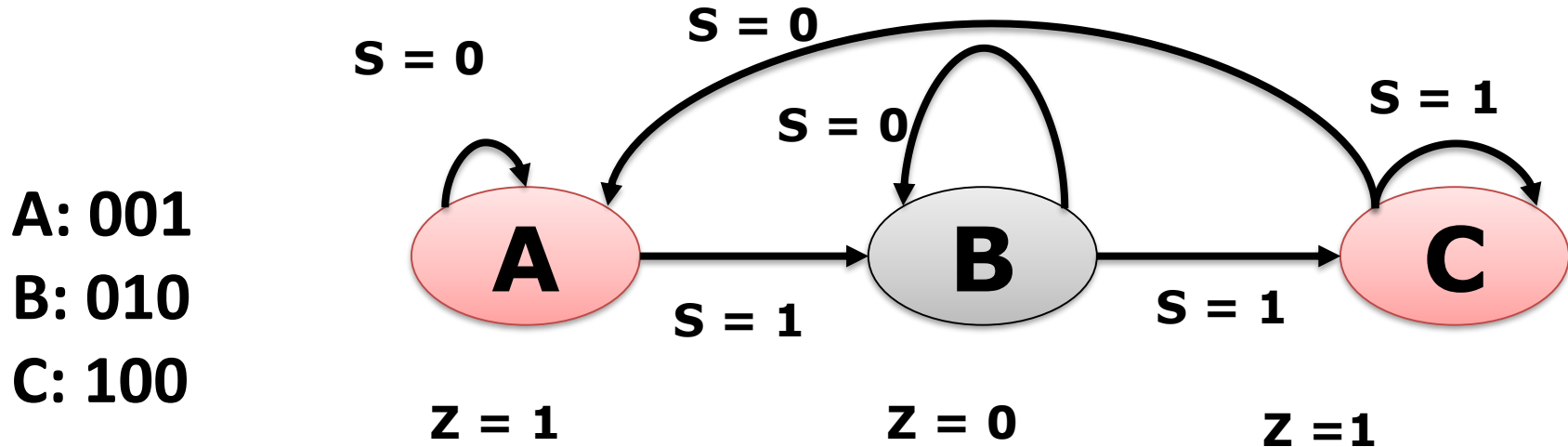
- The cost & delay of FSM implementation depends on encoding of symbolic states.
 - e.g., 4 states can be encoded in $4! = 24$ different ways
- There are more than $n!$ different encodings for n states.
 - **Exploration of all encodings is impossible**, therefore heuristics are used
- Heuristics Used
 - One-hot encoding, Minimum-bit change, Prioritized adjacency

State Encoding

- The cost & delay of FSM implementation depends on encoding of symbolic states.
 - e.g., 4 states can be encoded in $4! = 24$ different ways
- As **$n!$** different encodings for **n** states.
 - **Exploration of all encodings is impossible**, therefore heuristics are used
- Heuristics Used
 - One-hot encoding, Minimum-bit change, Prioritized adjacency

One-hot Encoding

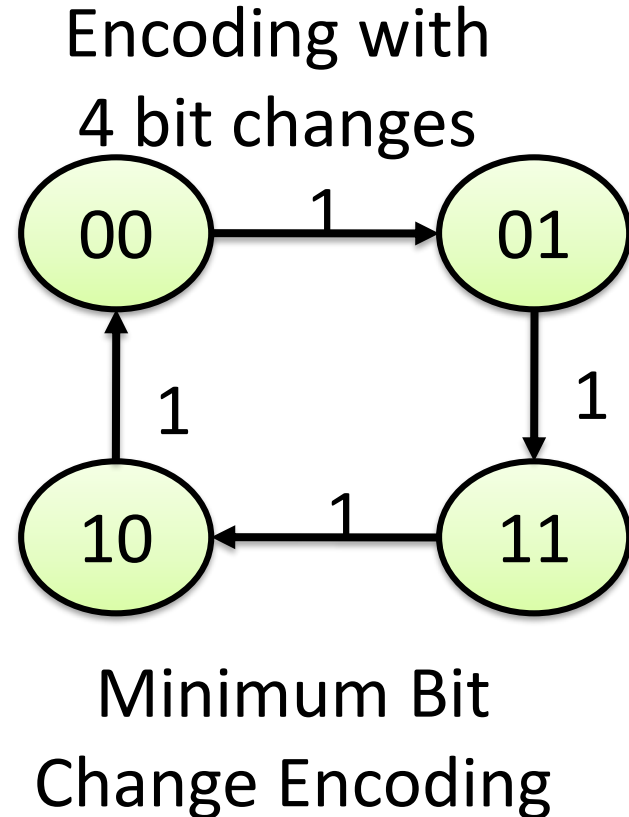
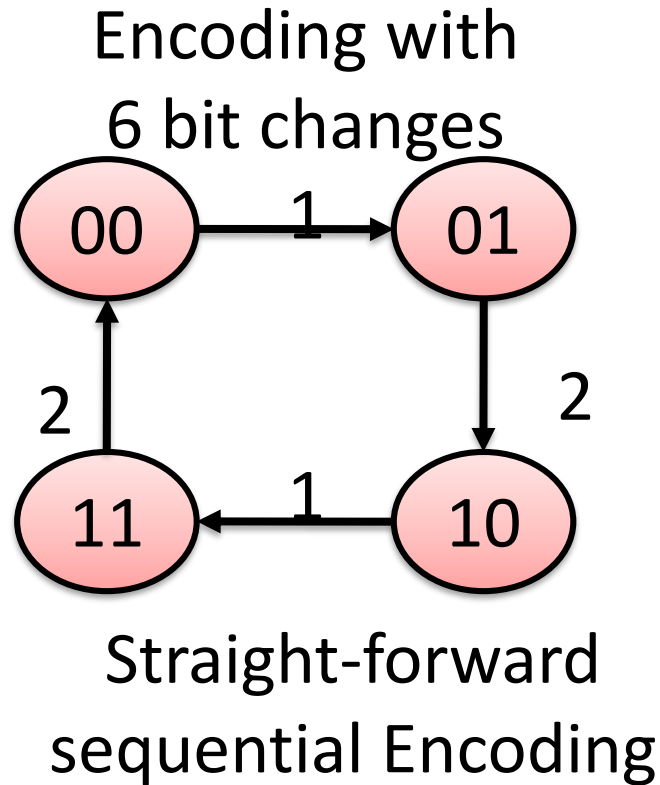
- Uses redundant encoding in which one flip-flop is assigned to each state.
- Each state is distinguishable by its own flip-flop having a value of 1 while all others have a value of 0.



Minimum-bit Change Encoding

- Assigns codes to states so that
 - The total number of **bit changes** for all state transitions is **minimized**.
- In other words, if every arc in the state diagram has a **weight**
 - That is equal to the **number of bits** by which the source and destination encoding **differ**
 - This strategy would select the one that **minimizes the sum** of all these weights.

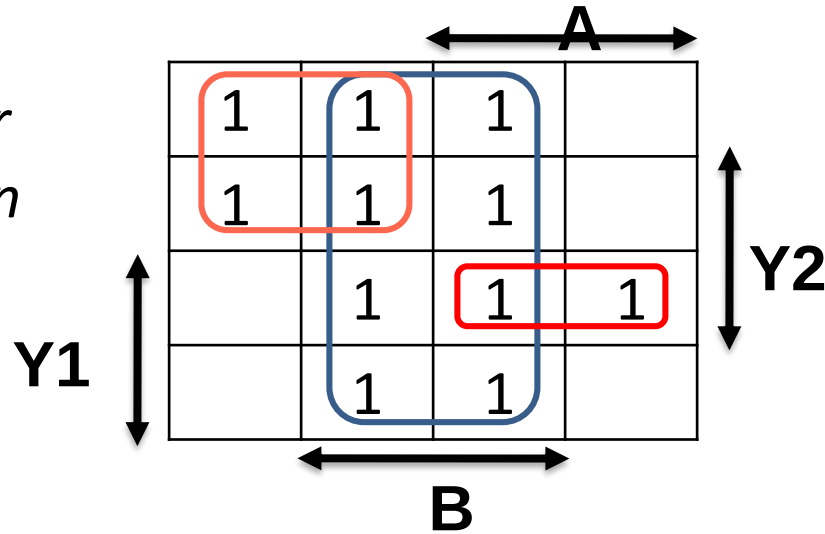
Minimum-bit Change Encoding



Idea of Adjacency of KMAP

- Inputs are A and B
- State variables are Y1 and Y2
- An output is $F(A, B, Y1, Y2)$ or $F(Y1, Y2)$
- A next state function is $G(A, B, Y1, Y2)$

*K map of
output function or
next state function*



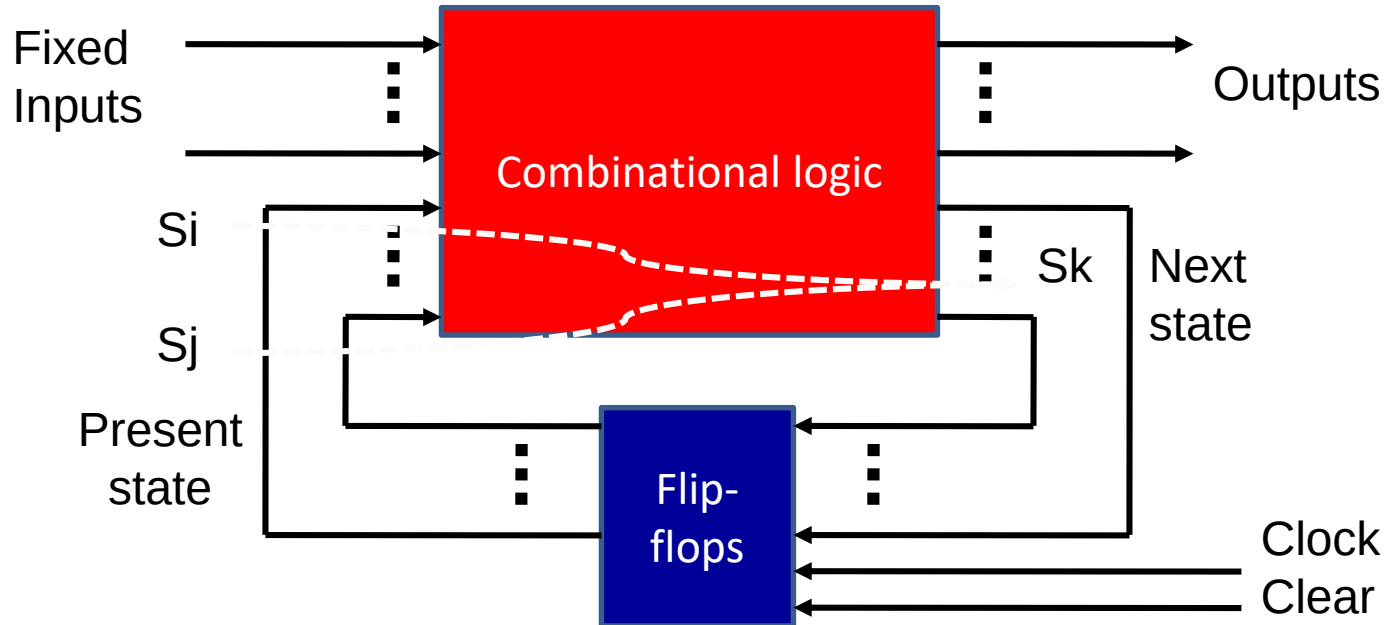
- Larger clusters produce smaller logic function.
- Clustered minterms differ in one variable.

Size of an Implementation

- Number of **product terms** determines
 - Number of **gates**
- Number **of literals** in a product term determines
 - Number of gate **inputs**, which is proportional to number of transistors.
- Hardware area proportional to
 - Total number of literals
- Examples of four minterm functions:
 - **F1** = $ABCD + A'B'C'D' + A'B'CD + A'B'CD$ has **16 literals**
 - **F2** = $ABC + A'C'D$ has **6 literals**

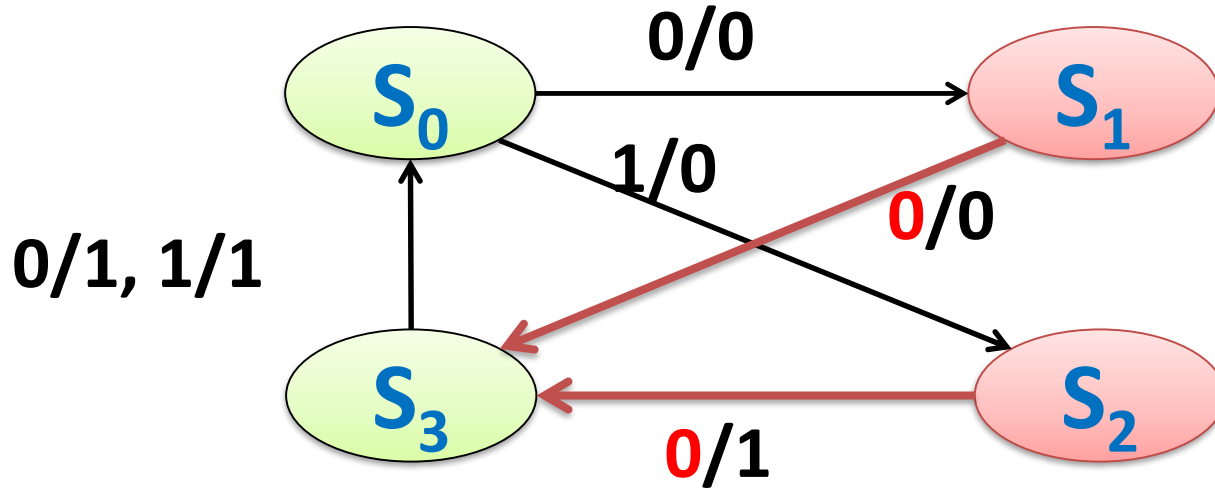
Rule 1 (Priority #1), Common Dest

States that **have** the **same next state** for some fixed input should be assigned logically **adjacent codes**.



Rule 1 (Priority #1), Common Dest

States that **have** the **same next state** for some fixed input should be assigned logically **adjacent codes**.



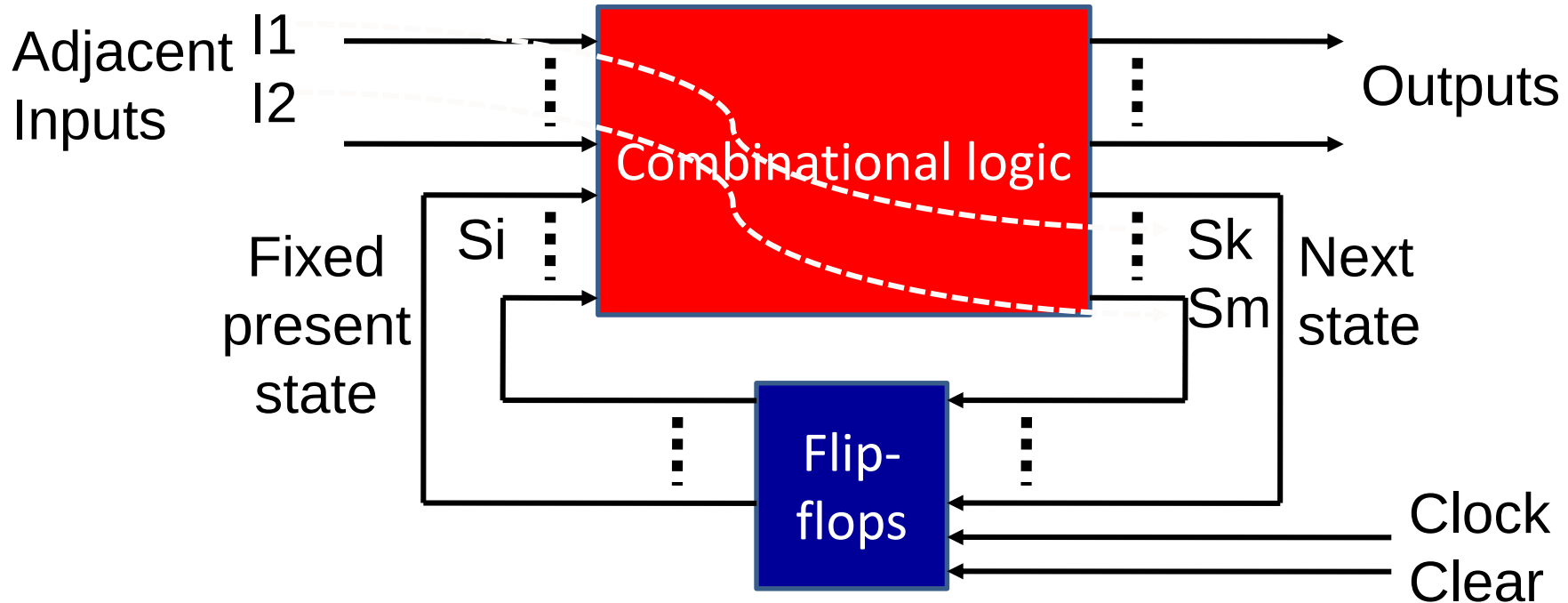
Rule #1: (S_1, S_2)

The input value of 0 will move both states into the same state S_3

→ Adj (S_1, S_2)

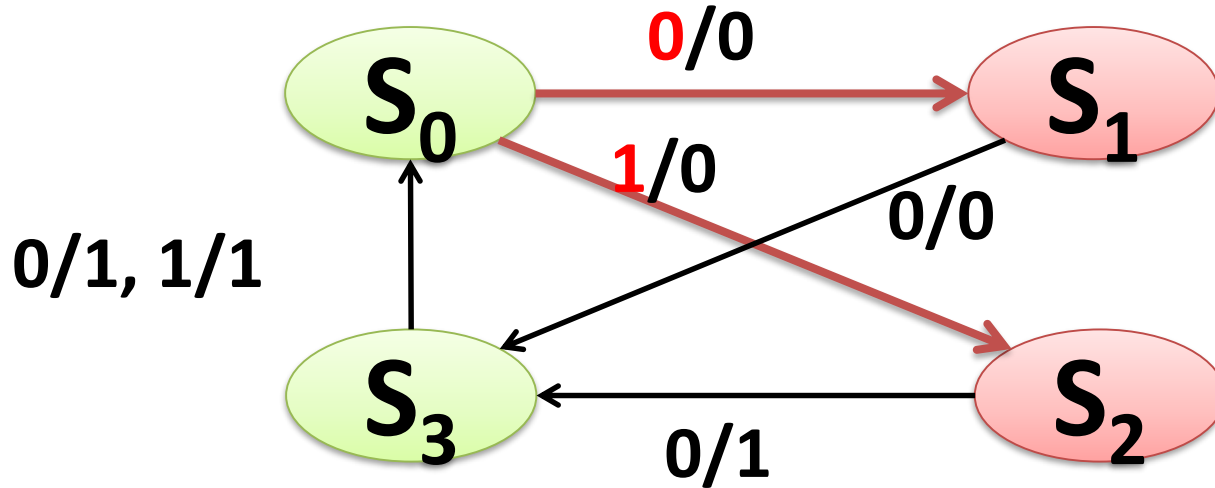
Rule 2 (Priority #2), A Common Source

States that are the next states of the same state under logically adjacent inputs, should be assigned logically adjacent codes.



Rule 2 (Priority #2), A Common Source

States that are the next states of the same state under logically adjacent inputs, should be assigned logically adjacent codes.

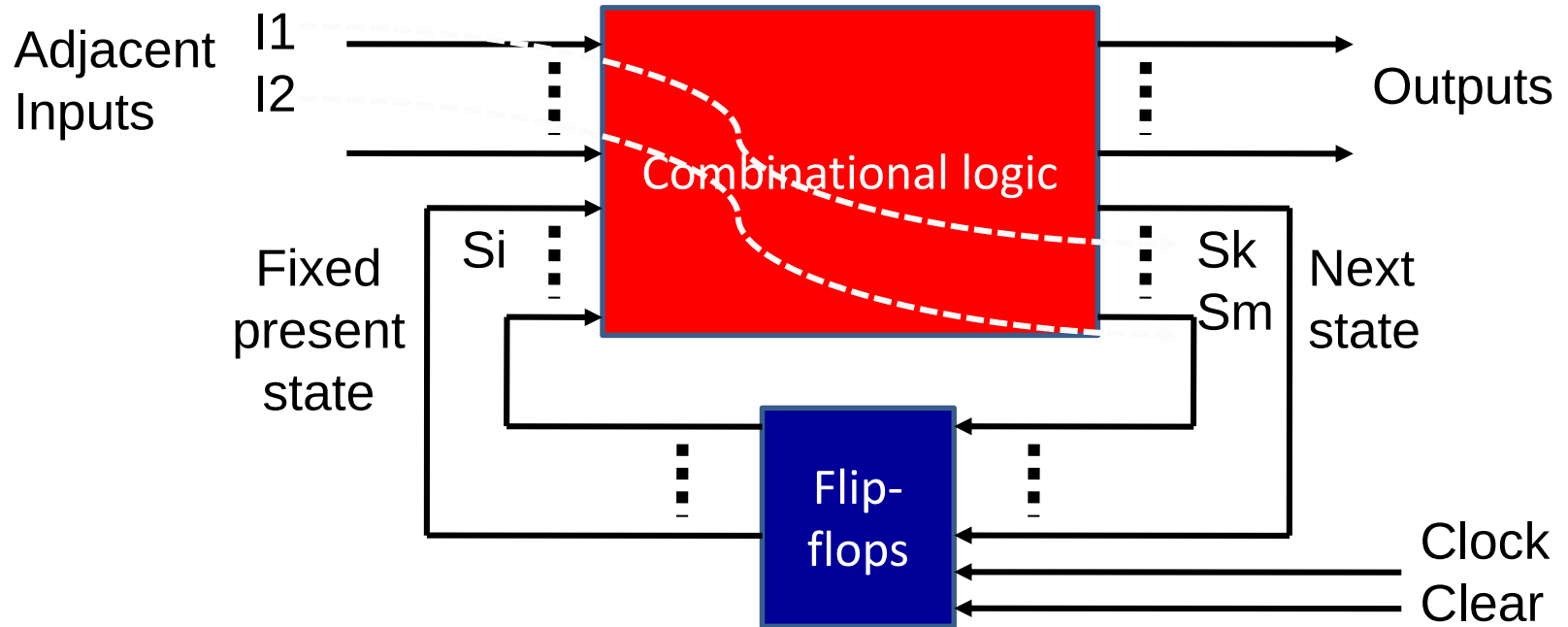


Rule #2: (S_1, S_2)

They are both next states of the state $S_0 \rightarrow \text{Adj}(S_1, S_2)$

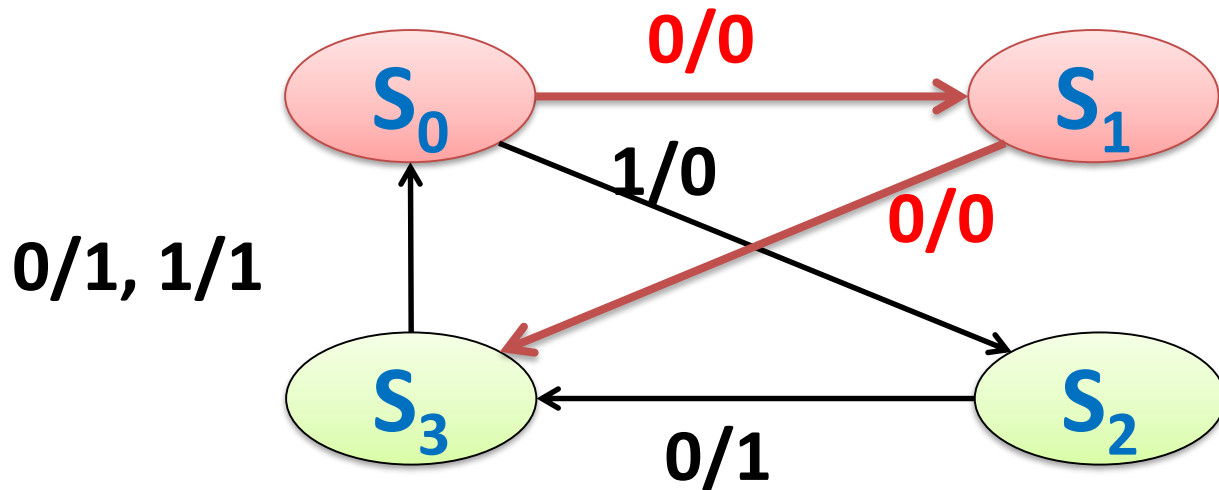
Rule 3 (Priority #3), A Common Output

States that have the same output value for the same input value, should be assigned logically adjacent codes.



Rule 3 (Priority #3), A Common Output

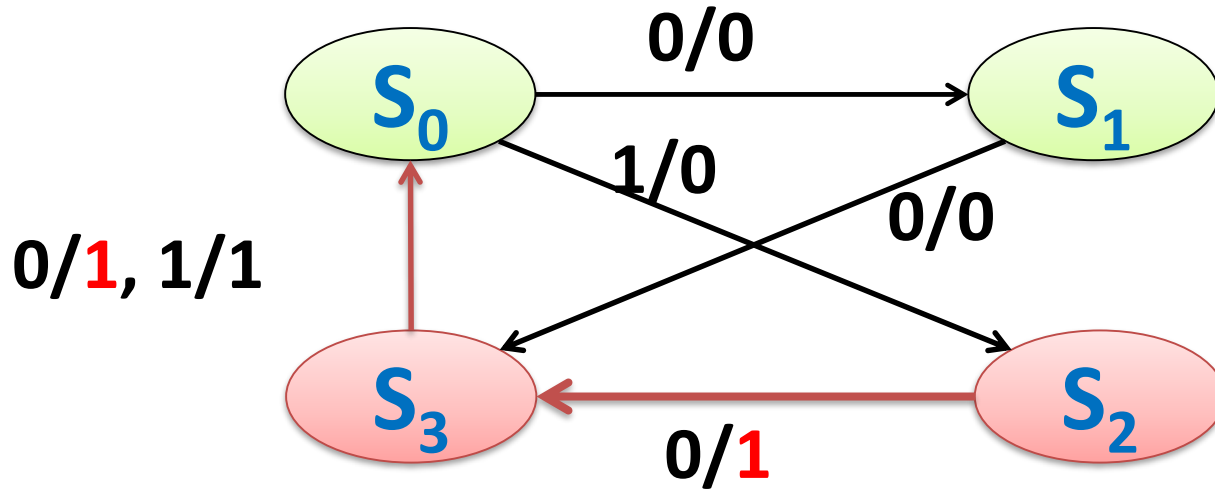
States that have the same output value for the same input value, should be assigned logically adjacent codes.



Rule #3: (S_0, S_1): states S_0 and S_1 have the same output value 0 for the same input value 0 $\rightarrow$ Adj (S_0, S_1)

Rule 3 (Priority #3), A Common Output

States that have the same output value for the same input value, should be assigned logically adjacent codes.



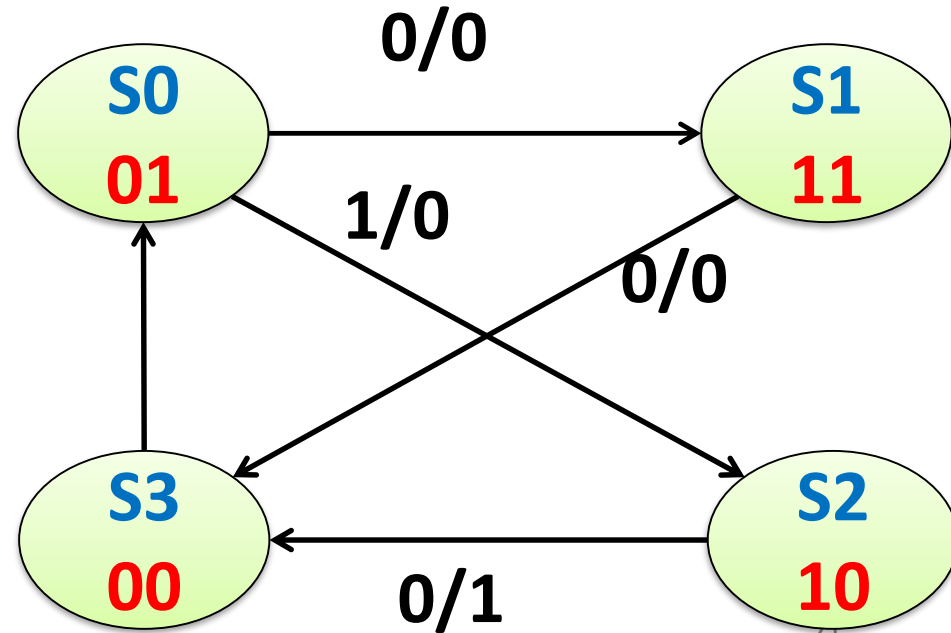
Rule #3: (S_2, S_3), states S_2 and S_3 have the same output value 1 for the same input value 0 $\rightarrow$ Adj (S_2, S_3)

Applying Rules 1,2 and 3

- Rule 1: $S1, S2 \implies HD(11, 10)=1$
- Rule 2 : $S1, S2$
- Rule 3: $S0, S1$ and $S2, S3$
 - $HD(S0,S1)=1$
 - $HD(S2,S3)=1$

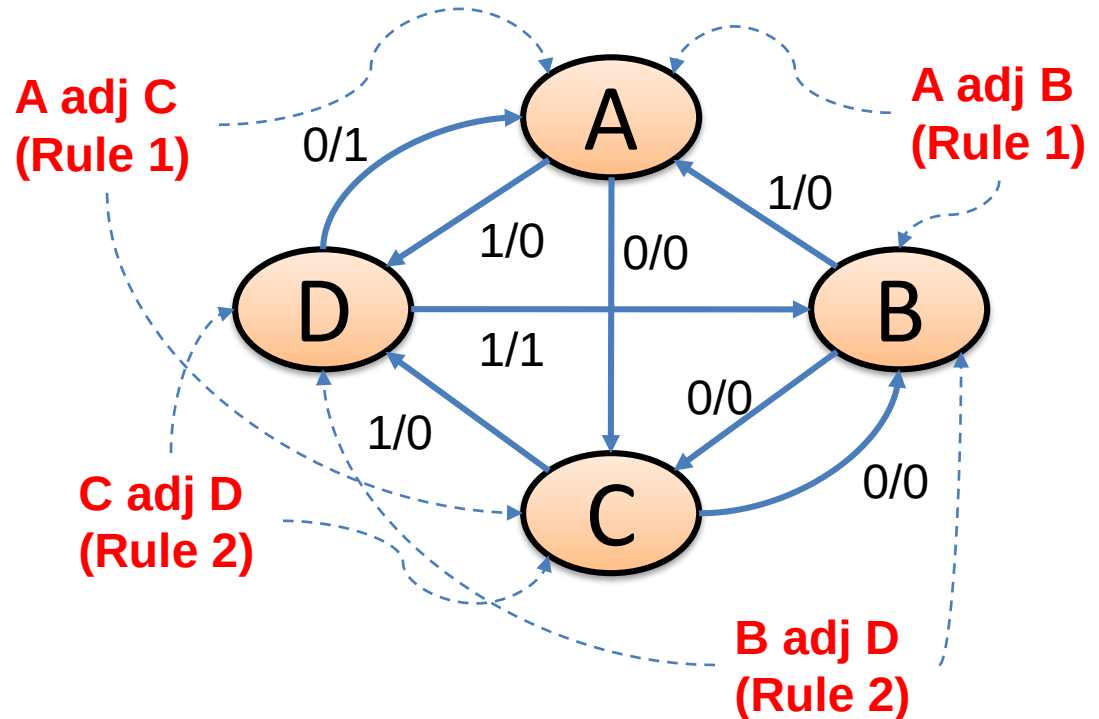
$0/1, 1/1$

- $HD(S0,S2)=2$
- $HD(S1,S3)=2$



Example of State Assignment

PS	NS, Z	
	Input X	
	0	1
A	C, 0	D, 0
B	C, 0	A, 0
C	B, 0	D, 0
D	A, 1	B, 1

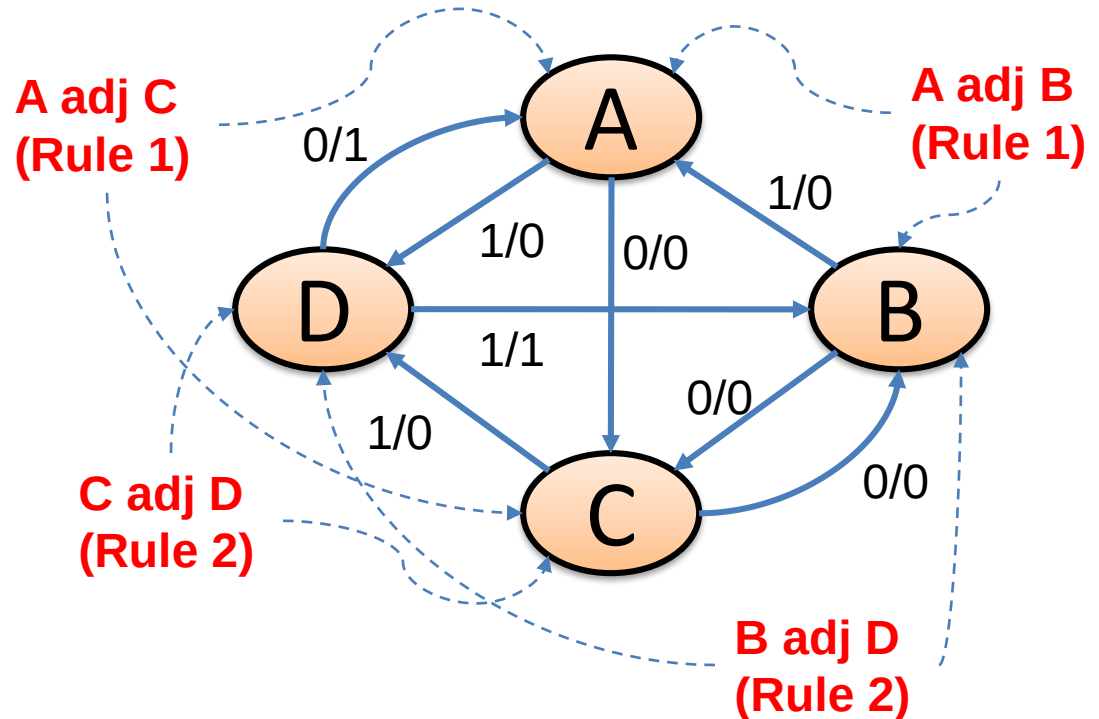


Rule 1: A and C goes to same next state D for same input 1

Rule 1: A and B goes to same next state C for same input 0

Example of State Assignment

PS	NS, Z	
	Input X	
	0	1
A	C, 0	D, 0
B	C, 0	A, 0
C	B, 0	D, 0
D	A, 1	B, 1

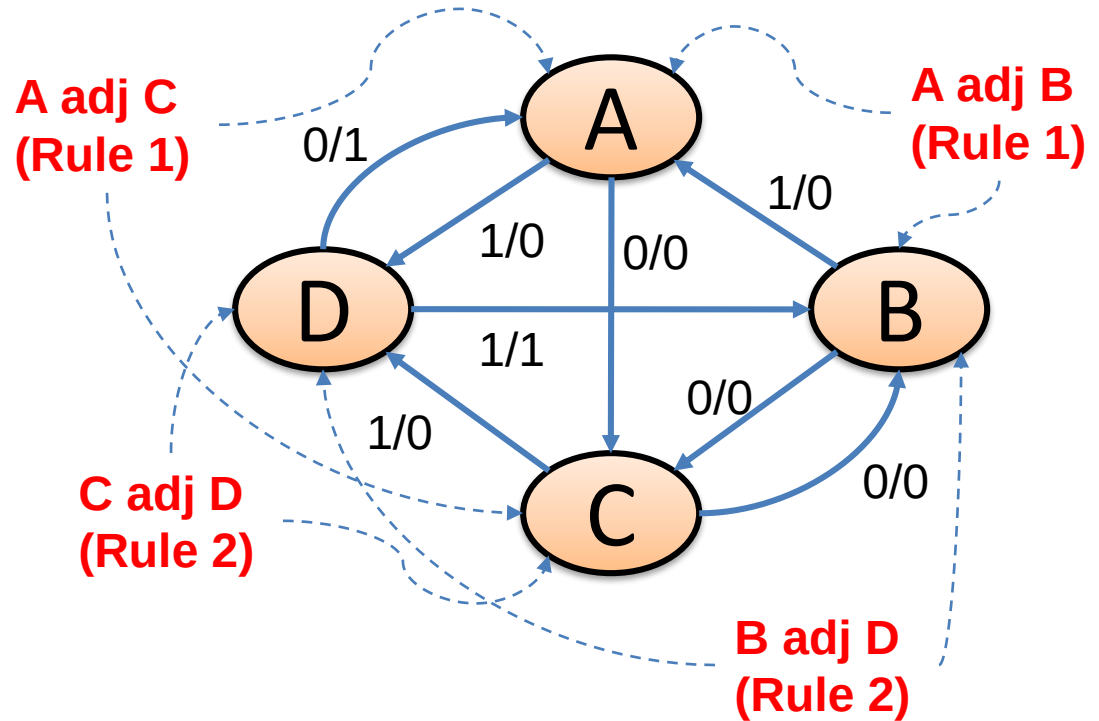


Rule 2: C and D are next state of A for input 0 and 1

Rule 2: B and D are next state of C for input 0 and 1

Example of State Assignment

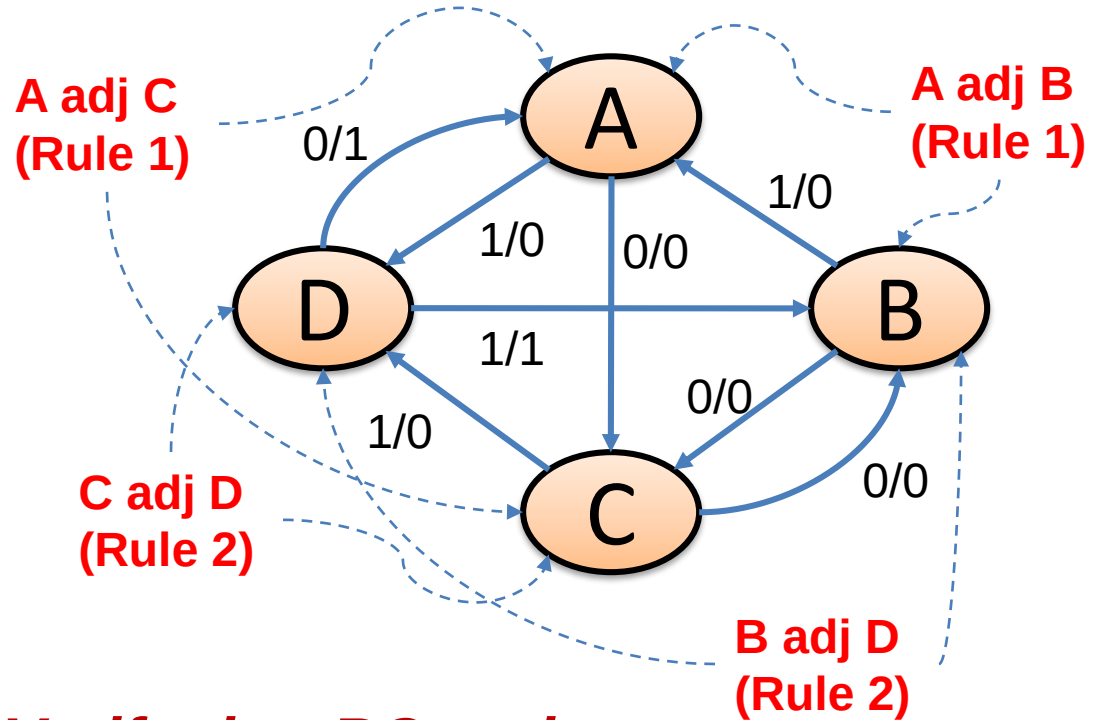
PS	NS, Z	
	Input X	
	0	1
A	C, 0	D, 0
B	C, 0	A, 0
C	B, 0	D, 0
D	A, 1	B, 1



Example of State Assignment

PS	NS, Z	
	Input X	
	0	1
A	C, 0	D, 0
B	C, 0	A, 0
C	B, 0	D, 0
D	A, 1	B, 1

	0	1
0	A	B
1	C	D



Verify that BC and AD are not adjacent.

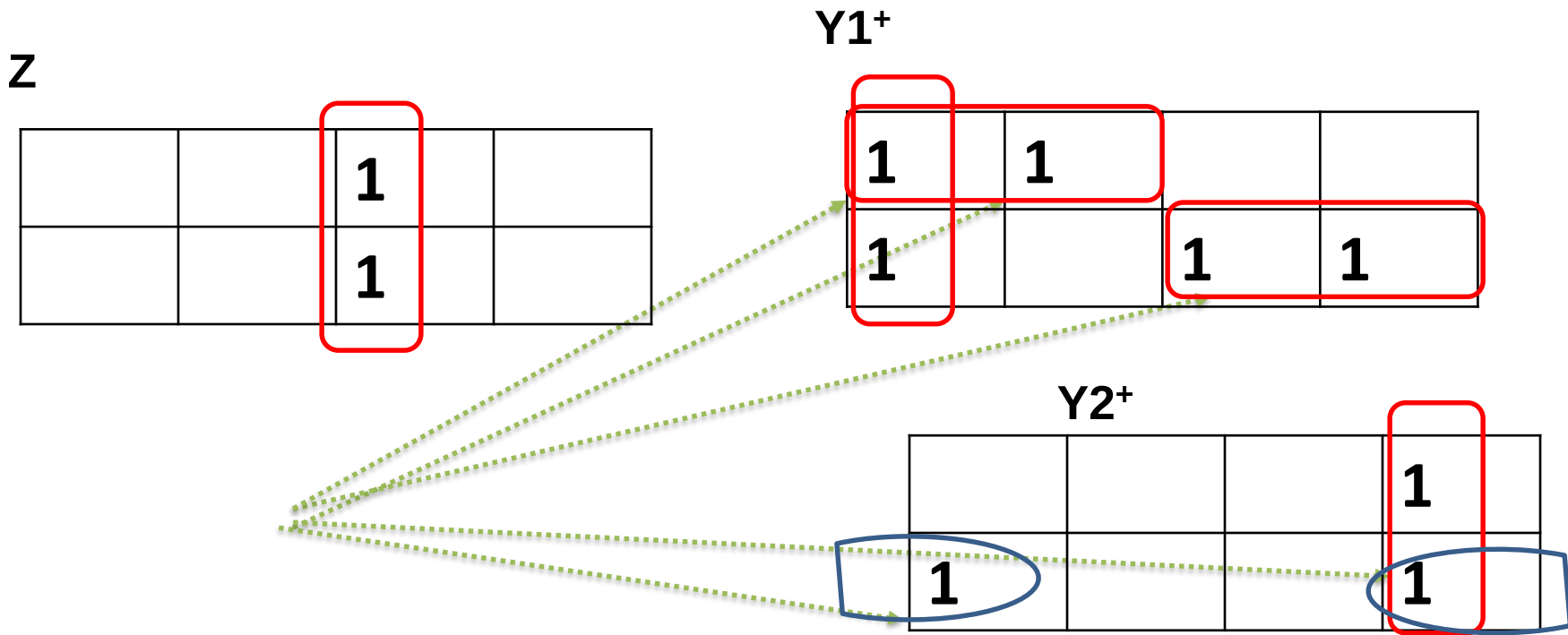
State Assignment #1

A = 00, B = 01, C = 10, D = 11

Present state Y1, Y2	Next state, output Y1*Y2*, Z	
	Input, X	
	0	1
A = 00	10 / 0	11 / 0
B = 01	10 / 0	00 / 0
C = 10	01 / 0	11 / 0
D = 11	00 / 1	01 / 1

Input X	Present state		Output Z	Next state	
	Y1	Y2		Y1 ⁺	Y2 ⁺
0	0	0	0	1	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	0	1	1
1	1	1	1	1	0

Logic for Optimum State Assignment



Result: 5 products, 10 literals.

Using an Arbitrary State Assignment

Present state Y1, Y2	Next state, output Y1*Y2*, Z	
	Input, X 0 1	
A = 00	11 / 0	10 / 0
B = 01	11 / 0	00 / 0
C = 11	01 / 0	10 / 0
D = 10	00 / 1	01 / 1

Input	Pre State		Output	Next state	
X	Y1	Y2	Z	Y1*	Y2*
0	0	0	0	1	1
0	0	1	0	1	1
0	1	0	1	0	0
0	1	1	0	0	1
1	0	0	0	1	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	0	1	0

Logic for Arbitrary State Assignment

Z (2,6)

			1	
			1	

Y1⁺

1	1		
1			1

Y2⁺

1	1	1	
			1

Current Result: 6 products,
14 literals.

Previous Result: 5 products,
10 literals.

ASM/FSMD or RTL Based Design

RTL Design

- Drawbacks of FSM
- $ASM = FSM + DP$
- ASM building blocks
- Modeling FSM in ASM

Reference Material

- Chapter 8 of Mano Book 6th Edition
 - Design at Register Transfer Level
 - Classic Example: Booth Multiplication
- Chapter 15 of Kumar Book
 - A. Anand Kumar, *Fundamentals of Digital Circuits* 3rd Edition, PHI. 2014

Drawbacks of FSM

- FSM for real systems:
 - Many inputs & many outputs -> awkward to list all of these as each transition arc.
 - On any given arc
 - Typically most inputs are don't care
 - Typically most outputs are unchanged from the settings in the previous state
 - Tedious & repetitive to list exhaustively

Drawbacks of FSM

- Not a clear structure for illustrating/designing control flow
- What about generic memory/data?
 - Do they really need to be part of the state? If we have many bits of data, this would lead to a huge state
 - **E.g. state diagram for counter or shift register is pointless**
 - **32 bit counter have 2^{32} states**

Drawbacks of FSM

- Some problems analogous to before
- Combinational: Circuit Design Using truth tables (TT); ok/easy for Small Problem
- Adders, Muxes – TT get out of hand
- Design 2 level circuit for a 32 bit Adder
 - 64 inputs and 33 outputs using TT method, worst case delay (2^{63} OR gate)
 - Worst case OR gate size or # of Product term $\Rightarrow 2^{N-1}-1$

Drawbacks of FSM: ASM Overview

- Some problems of Sequential Design
 - Small – state diagrams easy
 - Real, Data – state diagrams not helpful
 - **E.g. state diagram for 32 bit counter or shift register is pointless**
 - **32 bit counter have 2^{32} states**
- Solution is
 - ASM/FSMD/RTL Based Design

ASM/FSMD/RTL Based Design

- ASM/FSMD/RTL are similar terms
- ASM : Algorithmic State Machine
- FSMD : FSM with Data Path
- RTL Design : Register Transfer Level Design
- Define what work to be done at each clock cycle.

Algorithmic State Machine

Algorithmic State Machine –

- Another representation of a Finite State Machine
- Suitable for FSMs with
 - a larger number of inputs and outputs
- As compared to FSMs expressed using
 - state diagrams and
 - state tables.

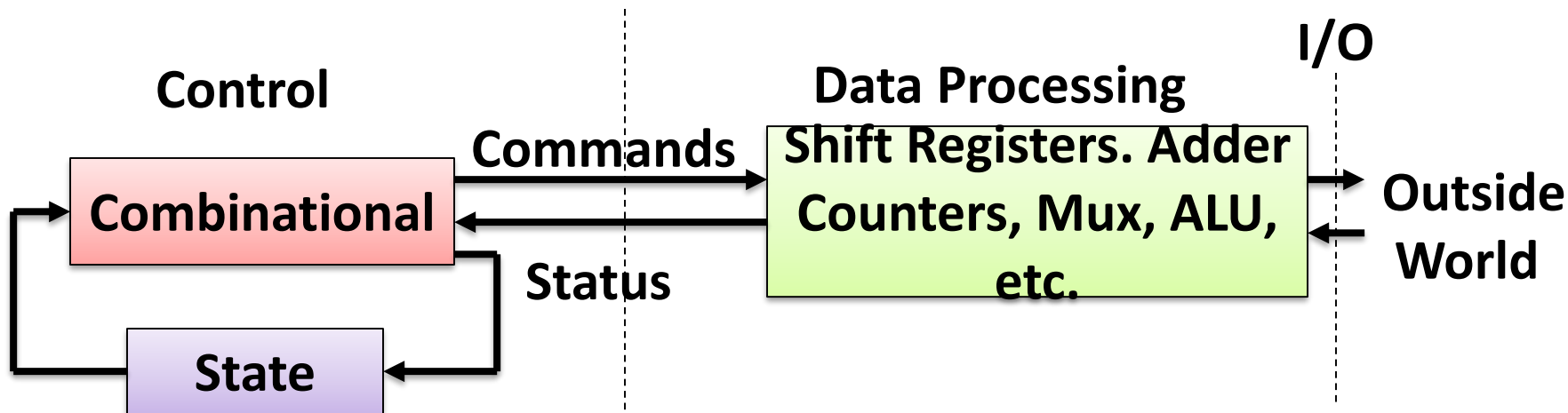
ASM Overview

- We need to separate controller & data processor
 - Controller – What actions need to be taken? What is fundamental operating mode?
 - Processor – Undertake the action. Manipulate the data

**The ultimate Goal : Design using Control Path +
Data Approach : RTL Design**

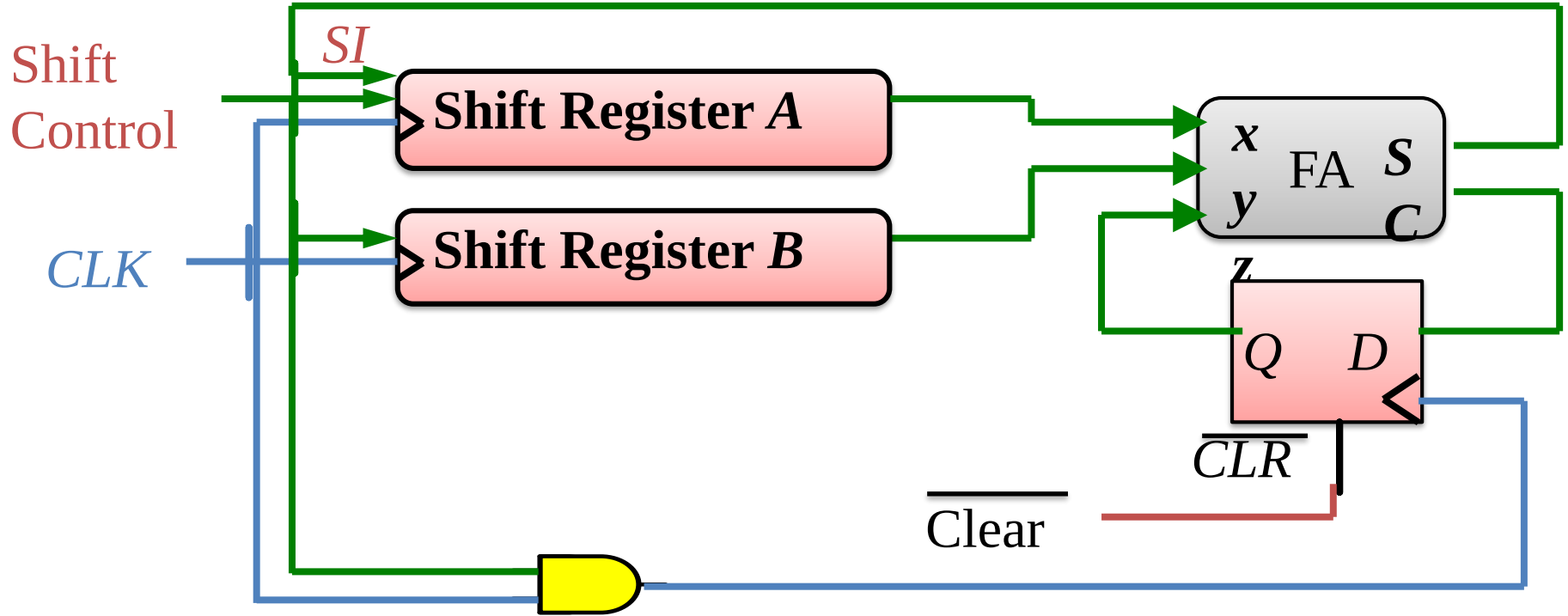
ASM Overview

- Control and data path interaction



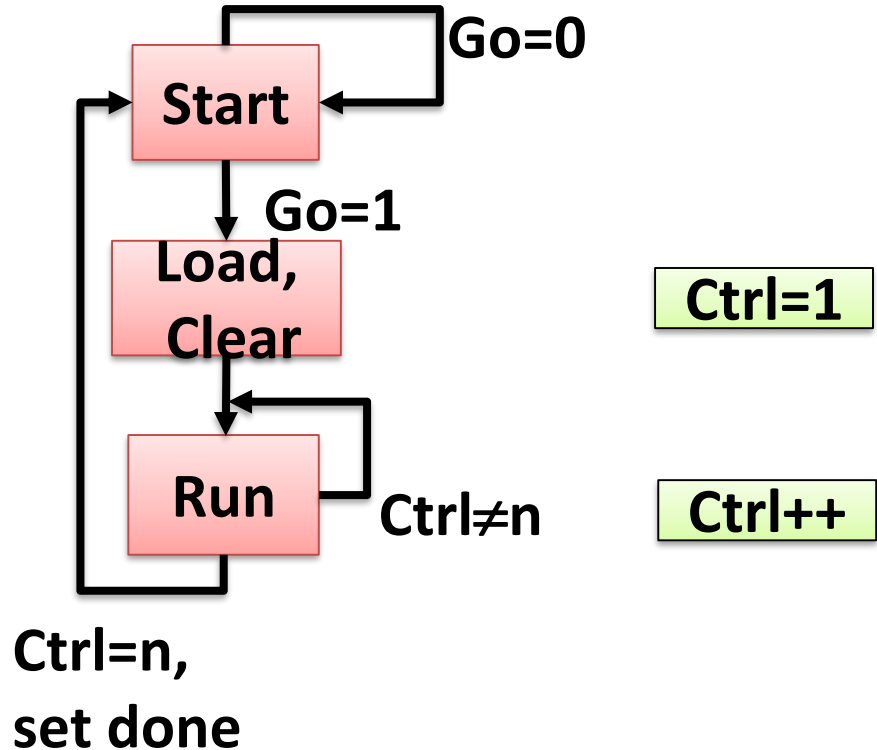
- Our circuit is now explicitly separated

Remember : Serial Addition



ASM Overview : High Level

- Ex. Serial Addition
- Control Part/Path



Serial Addition : Data Path

